TP1 : Partie 1

Combien de temps a duré le run ?



Où voit-on les logs de chaque étape ?
Dans le workflow de github accessible dans la section Action

Partie 2 :



Partie 3 :

Erreurs à la ligne 29 et 13 à la fin des tests:

```
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/maxime/Documents/GitOps/tasks-api
configfile: pyproject.toml
plugins: cov-7.1.0, platformdirs-4.14.3
collected 10 items

tests/test_api.py ...F.                                                                          [ 50%]
tests/test_logic.py ..F..                                                                        [100%]

=================================================== FAILURES ===================================================
_____________________________________ test_add_and_list_tasks ______________________________________

client = <FlaskClient <Flask 'app.main'>>

    def test_add_and_list_tasks(client):
        created = client.post("/tasks", json={"title": "  apprendre la ci "})
>       assert created.status_code == 201
E       assert 400 == 201
E        +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code

tests/test_api.py:29: AssertionError
___________________________________ test_is_valid_title_accepts_normal_title ___________________________________

    def test_is_valid_title_accepts_normal_title():
>       assert is_valid_title("Faire le TP1") is True
E       AssertionError: assert False is True
E        +  where False = is_valid_title('Faire le TP1')

tests/test_logic.py:13: AssertionError
=========================================== short test summary info ============================================
FAILED tests/test_api.py::test_add_and_list_tasks - assert 400 == 201
FAILED tests/test_logic.py::test_is_valid_title_accepts_normal_title - AssertionError: assert False is True
========================================= 2 failed, 8 passed in 0.30s ==========================================
```

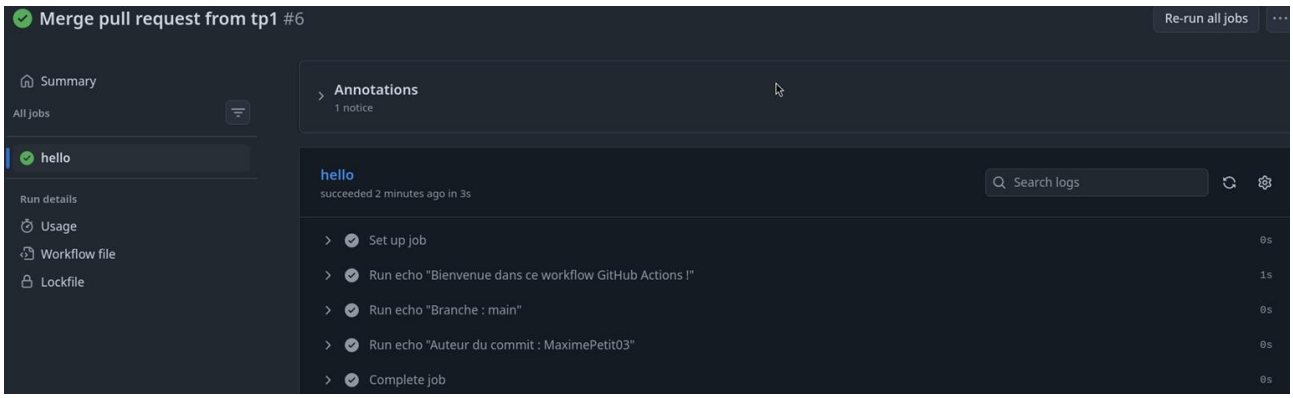
Correction :

```
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/maxime/Documents/GitOps/tasks-api
configfile: pyproject.toml
plugins: cov-7.1.0, platformdirs-4.14.3
collected 10 items

tests/test_api.py .....                                                                  [ 50%]
tests/test_logic.py .....                                                                [100%]

============================================== 10 passed in 0.28s ==============================================
```

Partie 4 :



Partie 5 :

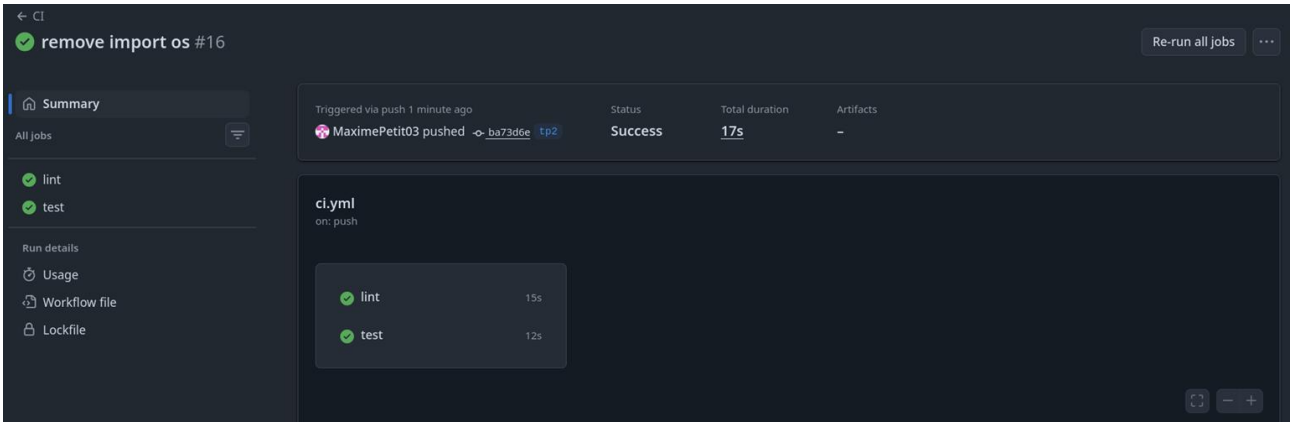


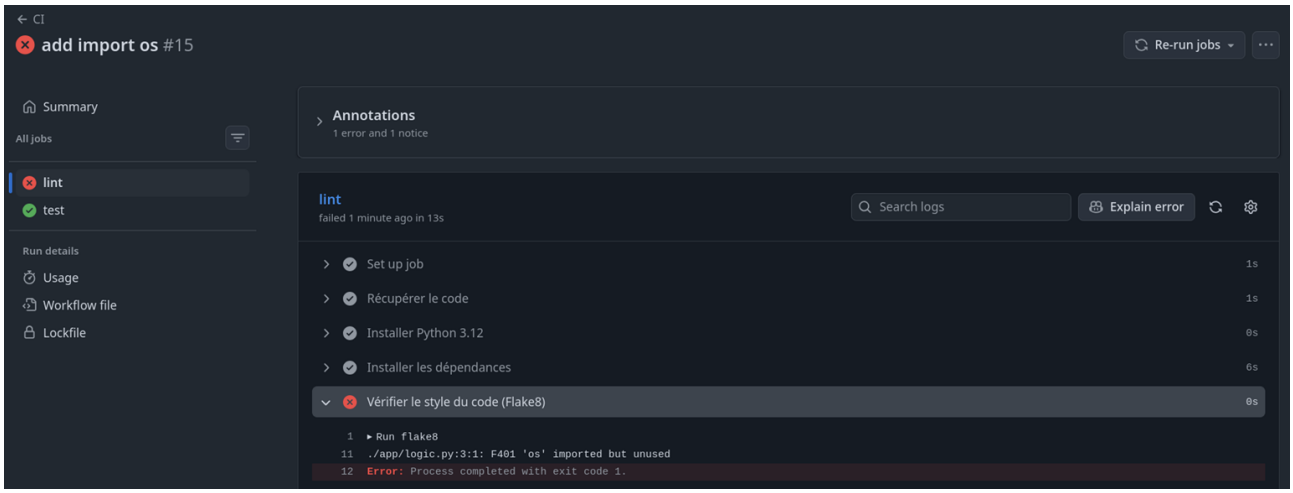
Partie 6 :



TP2 :

Partie 1 :

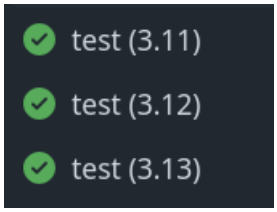


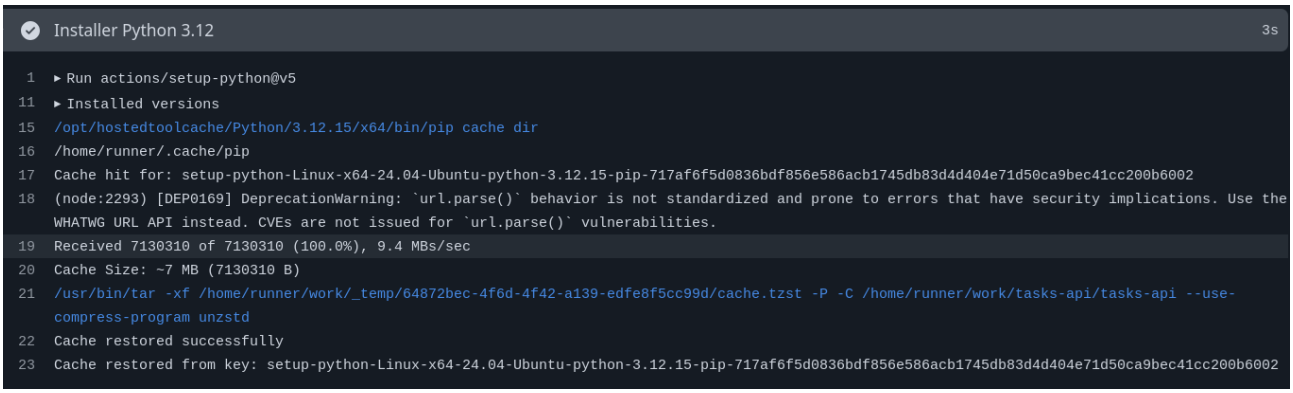


Quelle différence entre un linter (flake8) et un formateur (black) ? Un linter détecte les imports inutilisés les variables non définies et le formateur impose un style unique : espace, guillemets, retours à la ligne

Pourquoi séparer lint et test en deux jobs ? On sépare le lint et les tests en deux jobs pour les exécuter en parallèle, obtenir un retour rapide sur les erreurs de style et identifier immédiatement l'origine exacte d'un échec dans la CI

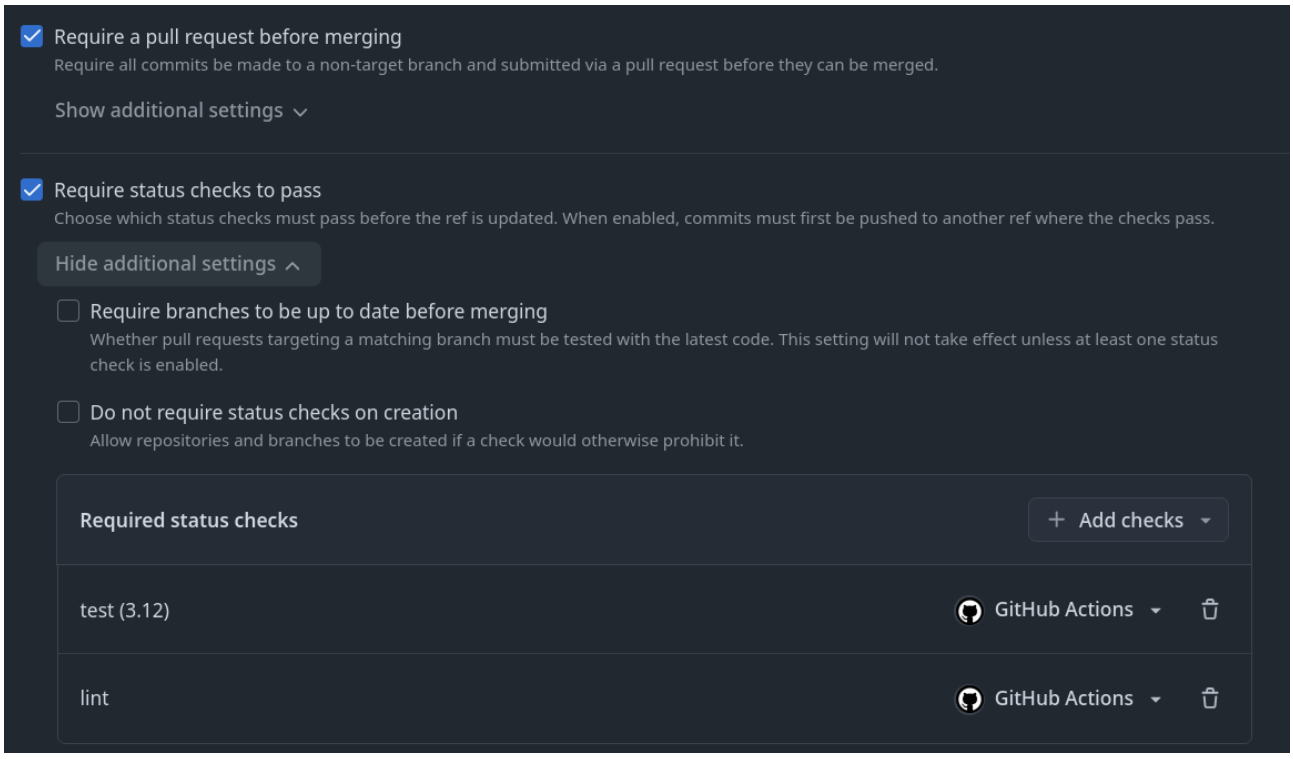
Partie 2 :

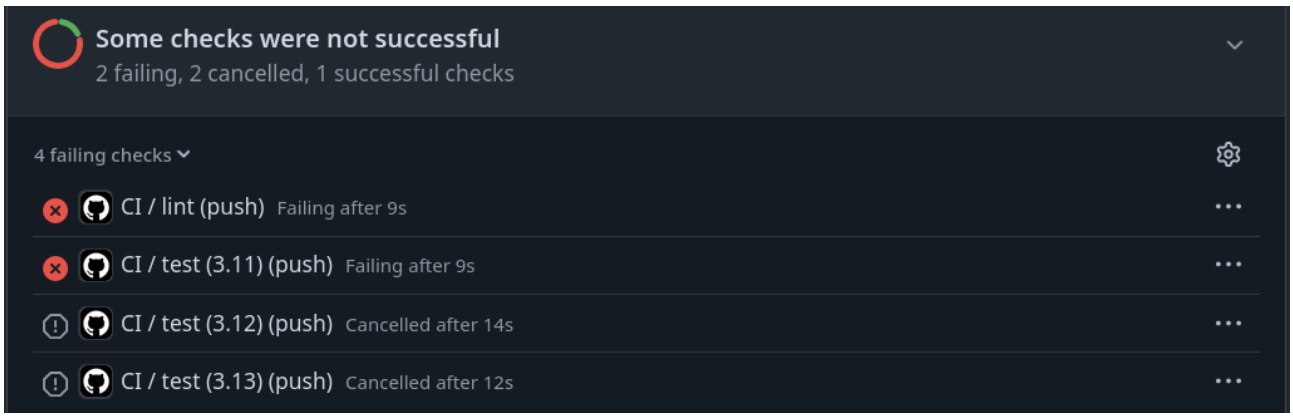




Questions : Pourquoi tester sur plusieurs versions de Python ? En cas de migration de notre api ou application
Que gagne-t-on avec le cache ? De la vitesse d'exécution
Un artefact, c'est quoi ? Un artefact est un fichier produit par un job CI/CD et sauvegardé sur GitHub pour être téléchargé ou réutilisé par d'autres étapes

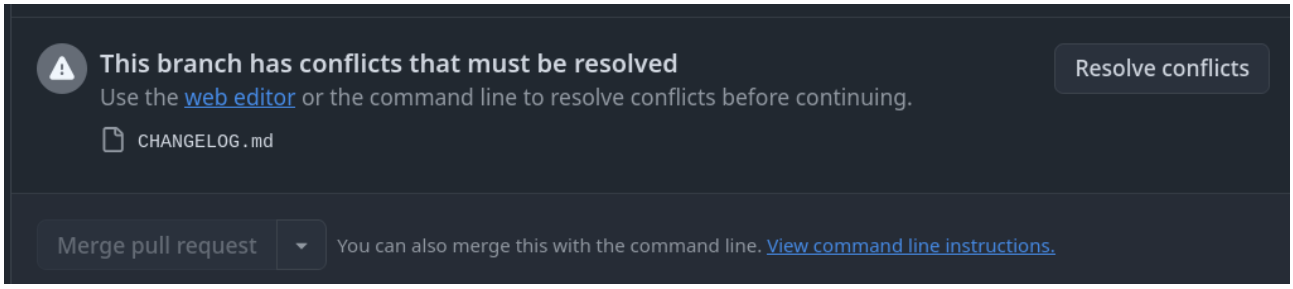
Partie 3 :

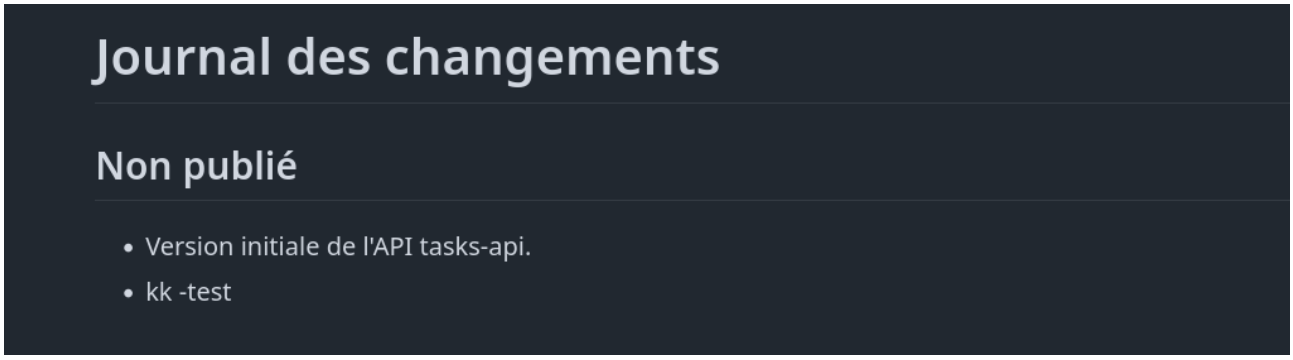




Question : Pourquoi interdire de pousser directement sur main ? Pour éviter de casser sans prévenir le code principale il faut d'abord vérifier que rien ne casse puis pousser ensuite

Partie 4 SEUL:





PS : kk est la première ligne et test est la 2e c'est juste une erreur de synthaxe de ma part

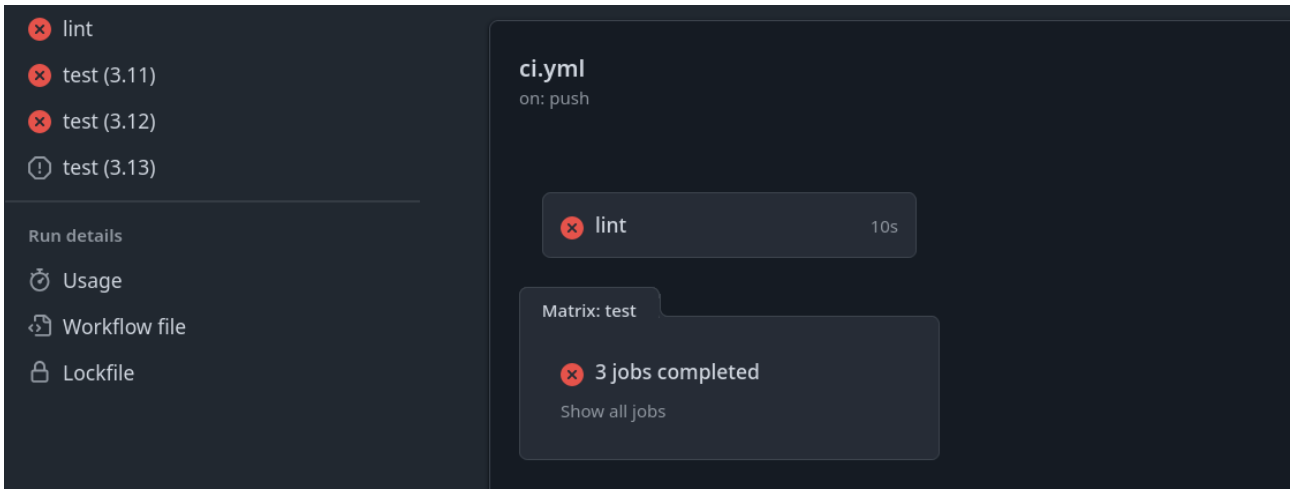
Questions : Pourquoi Git ne peut-il pas résoudre ce conflit seul ? Parce que ce sont les devs qui ont créer le/les conflits et donc ceux sont les seuls à savoir quoi garder et quoi supprimer
Qu'est-ce qui a vérifié le code après la résolution ? Workflow GitHub Actions

Partie 5 :



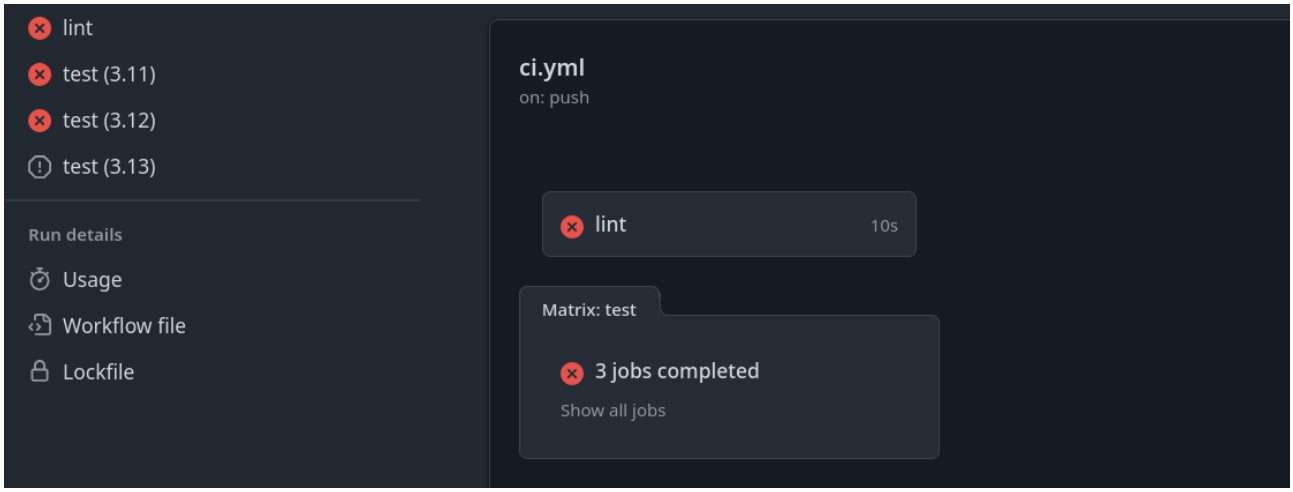
Questions : Pourquoi GitHub masque-t-il le secret ? GitHub retranscrit la valeur du secret par trois * dans les logs pour empêcher toute fuite accidentelle d'identifiants confidentiels à l'écran
Que se passe-t-il si on le commite dans le dépôt ? Le secret devient immédiatement visible dans l'historique Git et risque d'être détecté puis exploité par des bots malveillants, il faut donc le révoquer
Secret ou variable : comment choisir ? Utilisez un secret pour toute donnée confidentielle à masquer comme une clé API ou un mot de passe, et une variable pour les éléments de configuration qui ne sont pas sensible comme une variable d'environnement

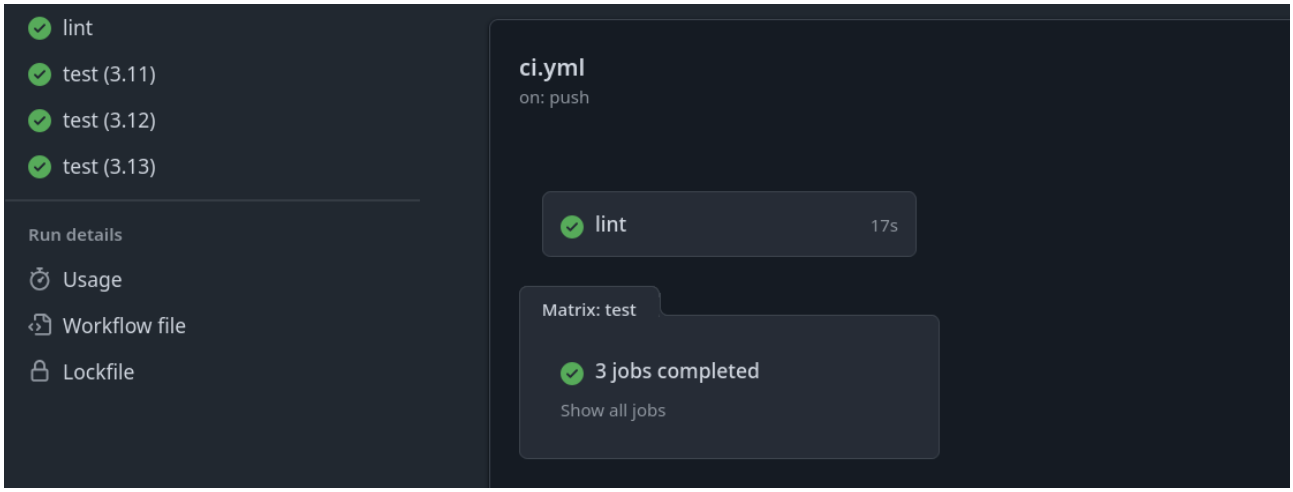
Partie 6 :



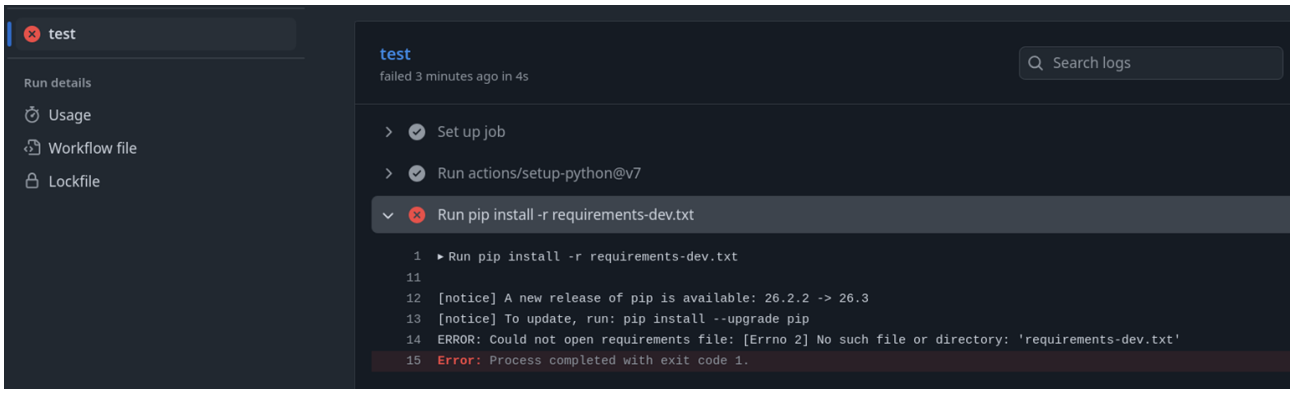
Questions : Pour chaque pipeline : quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

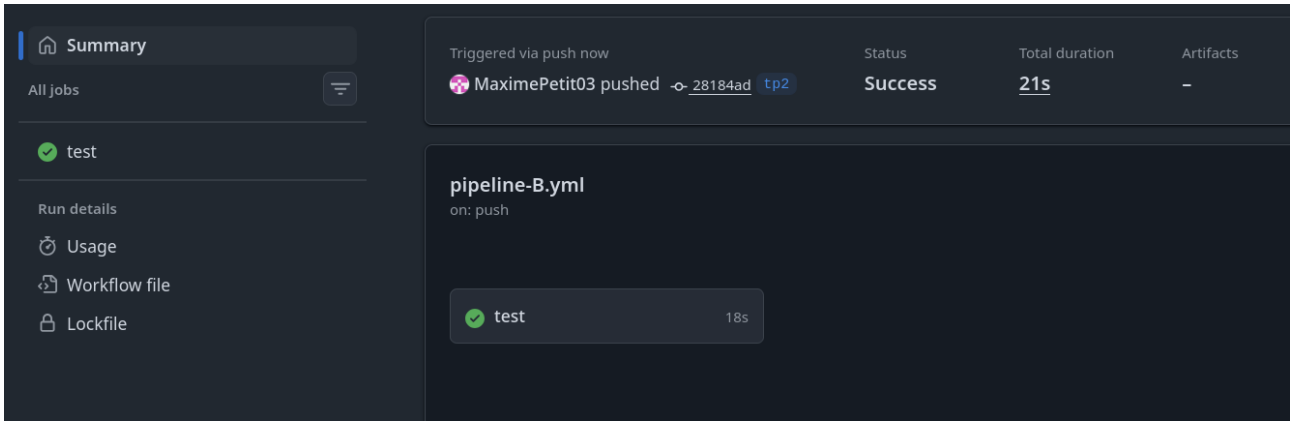
Pipeline A : le bug était un problème d'indentation à la dernière ligne. Je l'ai trouvé en lisant le fichier. Pour l'éviter il faut se relire et vérifier le fichier avec des outils (IA...)



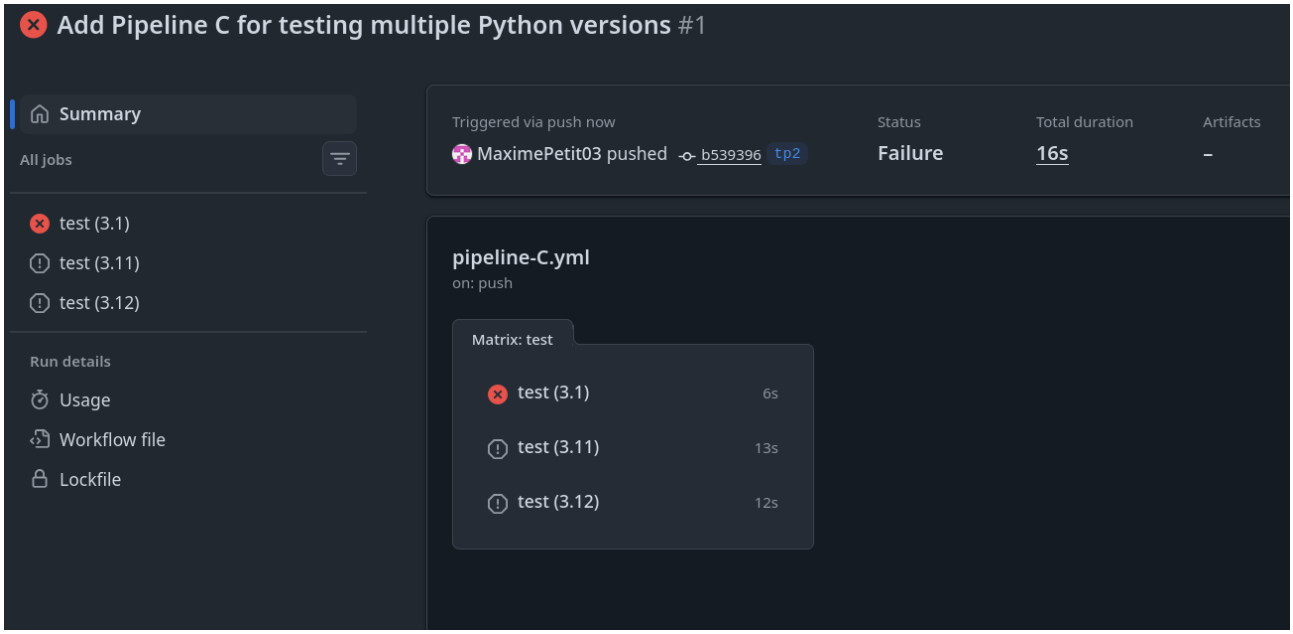


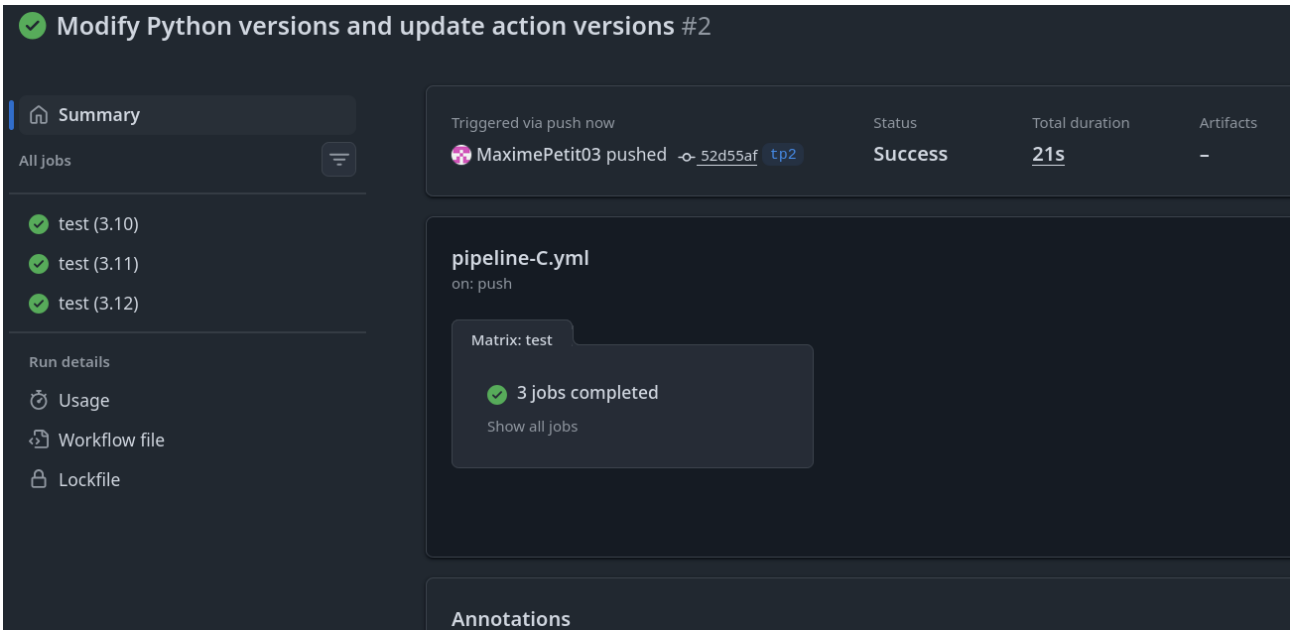
Pipeline B : Pas de checkout dans steps. Trouvé grâce au refus du workflow du Github Action. Pour l'éviter à l'avenir on peut se faire une checklist de ce qu'on doit mettre dans le fichier yaml





Pipeline C : Problème de guillemets pour les versions. J'ai trouvé l'erreur en lisant le fichier ou en regardant le workflow. Pour éviter l'erreur à l'avenir je dois mieux lire mon fichier avant de commit





TP3 :

Partie 1 :

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ docker build -t tasks-api:local .
[+] Building 40.1s (11/11) FINISHED                                                                                 docker:default
 => [internal] load build definition from Dockerfile                                                                         0.0s
 => => transferring dockerfile: 1.29kB                                                                                       0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                          1.9s
 => [internal] load .dockerignore                                                                                            0.0s
 => => transferring context: 181B                                                                                            0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031a  29.9s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab  0.0s
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e 251B / 251B                                   0.2s
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3 12.12MB / 12.12MB                            11.7s
 => => sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35 1.29MB / 1.29MB                               2.7s
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5 29.86MB / 29.86MB                            27.9s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5                                    1.2s
 => => extracting sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35                                    0.1s
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3                                    0.6s
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e                                    0.0s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 39.82kB                                                                                         0.0s
 => [2/6] WORKDIR /app                                                                                                       0.1s
 => [3/6] COPY requirements.txt .                                                                                            0.0s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                                 4.4s
 => [5/6] COPY . .                                                                                                           0.1s
 => [6/6] RUN useradd -m appuser && chown -R appuser:appuser /app                                                            0.3s
 => exporting to image                                                                                                       3.2s
 => => exporting layers                                                                                                      1.3s
 => => exporting manifest sha256:b16a9feaa612d88b330581c880d5adb92522c66823dcb47d66e9008628e95b17                            0.0s
 => => exporting config sha256:8307c5a30a2b7ecb1e452448e7c83b4c1beaeca72e6ccf22b28d6670575fd07a                              0.0s
 => => exporting attestation manifest sha256:38d0cacbe4b02651cda61e69a3c20f42ef95035fa51a0e8809112f1d8977fe59                0.0s
 => => exporting manifest list sha256:d66ad78786373738ce57b6065dce99747a84b85325b868c156e1055fe83500eb                       0.0s
 => => naming to docker.io/library/tasks-api:local                                                                           0.0s
 => => unpacking to docker.io/library/tasks-api:local                                                                        1.7s
```

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8000/health
{"status":"ok"}
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8000/version
{"version":"dev"}
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Questions : Différence entre une image et un conteneur ? Une image est une recette figée et le conteneur résultat de cette recette
À quoi sert le .dockerignore ? Il sert à savoir qu'est ce qui ne doit pas être envoyé à docker pendant le build

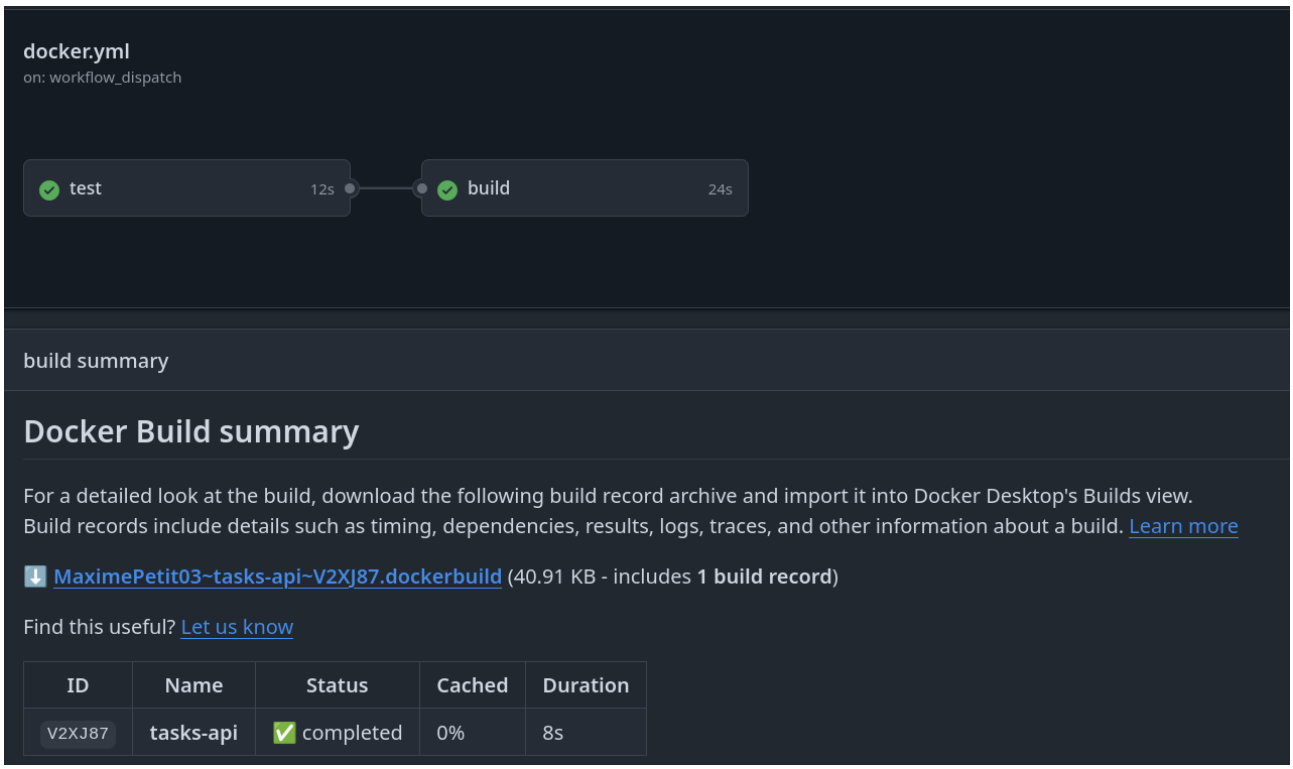
Partie 2 :

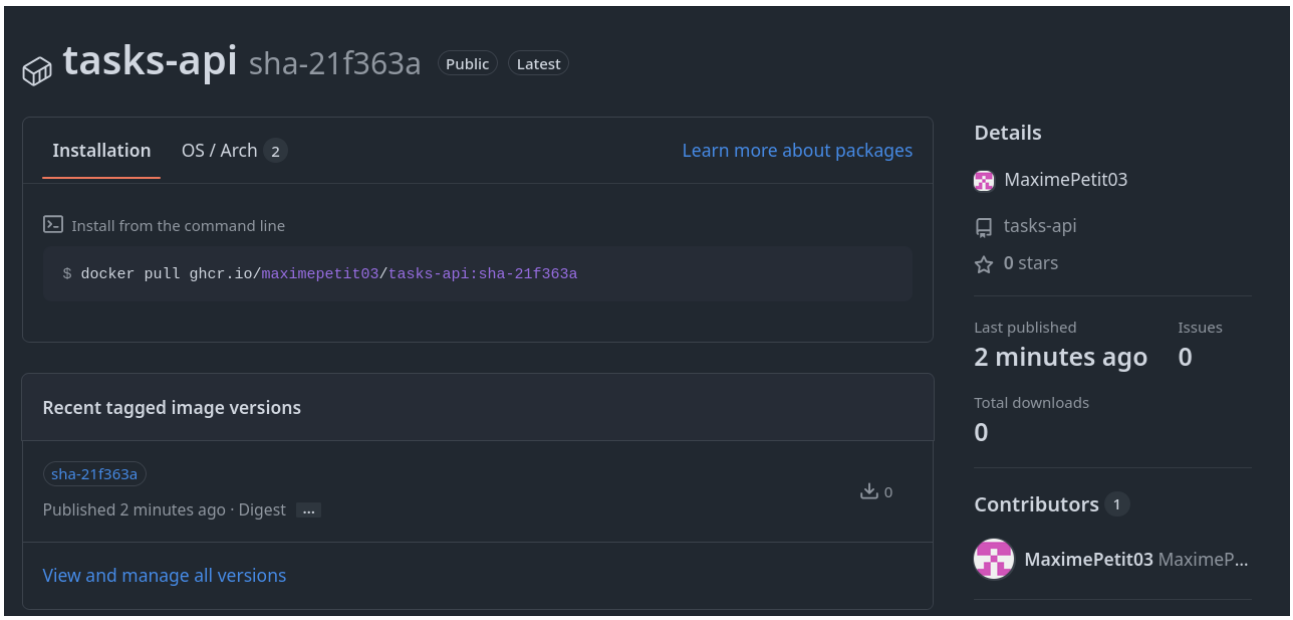
```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ docker compose up --build
[+] Building 1.3s (13/13) FINISHED
 => [internal] load local bake definitions                                                                                                0.0s
 => => reading from stdin 522B                                                                                                            0.0s
 => [internal] load build definition from Dockerfile                                                                                      0.0s
 => => transferring dockerfile: 1.29kB                                                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                                       0.9s
 => [internal] load .dockerignore                                                                                                         0.0s
 => => transferring context: 181B                                                                                                         0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30                  0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30                  0.0s
 => [internal] load build context                                                                                                         0.0s
 => => transferring context: 1.16kB                                                                                                       0.0s
 => CACHED [2/6] WORKDIR /app                                                                                                             0.0s
 => CACHED [3/6] COPY requirements.txt .                                                                                                  0.0s
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                                       0.0s
 => CACHED [5/6] COPY . .                                                                                                                 0.0s
 => CACHED [6/6] RUN useradd -m appuser && chown -R appuser:appuser /app                                                                  0.0s
 => exporting to image                                                                                                                    0.1s
 => => exporting layers                                                                                                                   0.0s
 => => exporting manifest sha256:2e7f31befc88510a93477b3805b057b78d0e50032df3917c1598ad32d848ae45                                         0.0s
 => => exporting config sha256:e01fe3410e4d625d141c0267cf2d040823546af0ea77392e37e590f764f3f9ee                                           0.0s
 => => exporting attestation manifest sha256:cd36f04a76a2c1bd9c9e4eda35296b68f7d5ca9b5cb9588ad3f621571aaa430a                             0.0s
 => => exporting manifest list sha256:56b1322c309351981f5a38b63e2cb358bce7506202ab0f874789eec421c29206                                    0.0s
 => => naming to docker.io/library/tasks-api-api:latest                                                                                   0.0s
 => => unpacking to docker.io/library/tasks-api-api:latest                                                                                0.0s
 => resolving provenance for metadata file                                                                                                0.0s
[+] up 1/1
 ✔ Image tasks-api-api Built                                                                                                              1.4s
Attaching to api-1
api-1  | [2026-11-04 09:21:46 +0000] [1] [INFO] Starting gunicorn 26.2.0
api-1  | [2026-11-04 09:21:46 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
api-1  | [2026-11-04 09:21:46 +0000] [1] [INFO] Using worker: sync
api-1  | [2026-11-04 09:21:46 +0000] [7] [INFO] Booting worker with pid: 7
api-1  | [2026-11-04 09:21:47 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
```

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8000/version
{"version":"local"}
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Question : Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ? Décrire le lancement dans un fichier permet de figer la configuration de l'application sous forme de code, rendant son exécution reproductible, partageable et utilisable en une seule commande courte

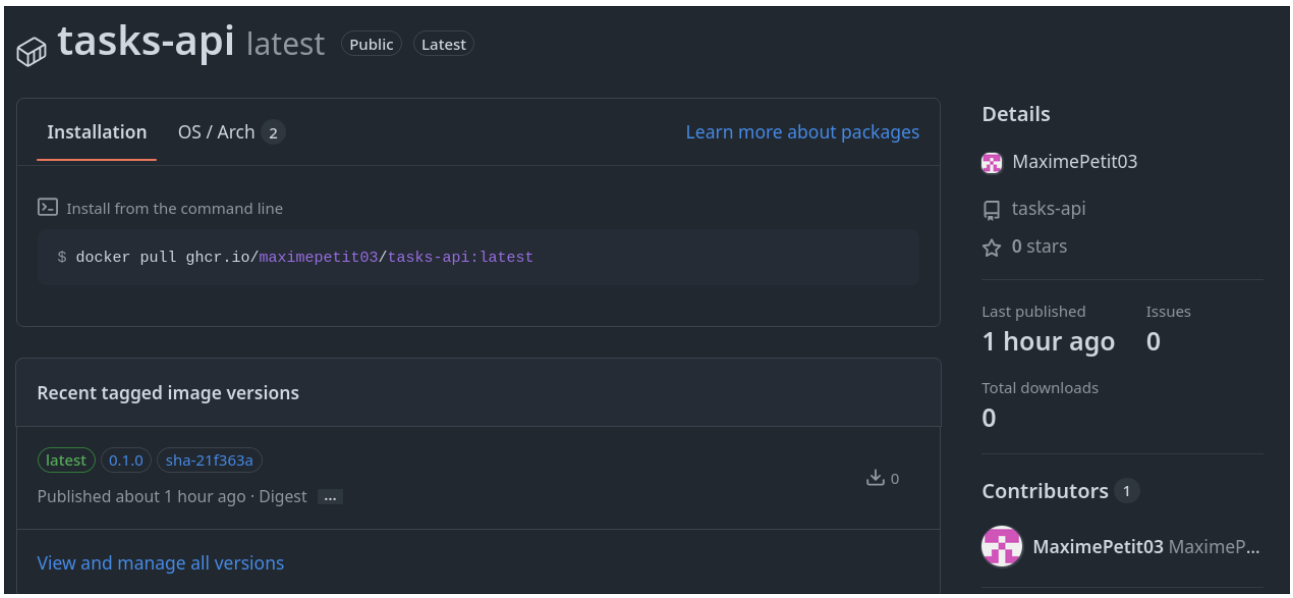
Partie 3 :





Questions : Pourquoi build attend-il test ? Pour éviter de donner du code cassé
Pourquoi éviter le tag latest en production ? Parce qu'on ne sait jamais dans quelle version il est

Partie 4 :

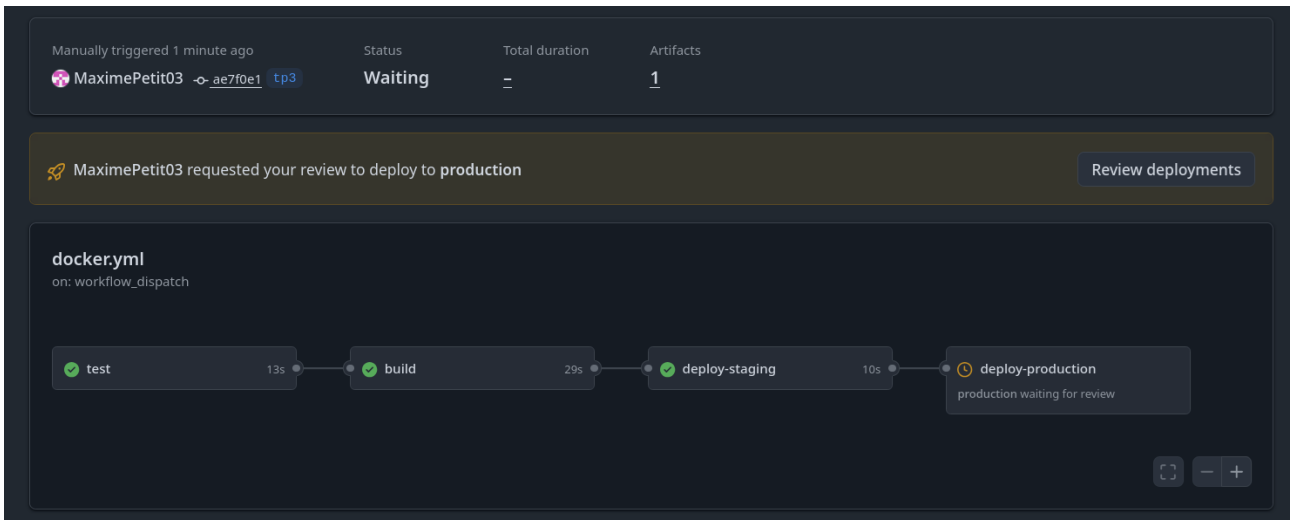


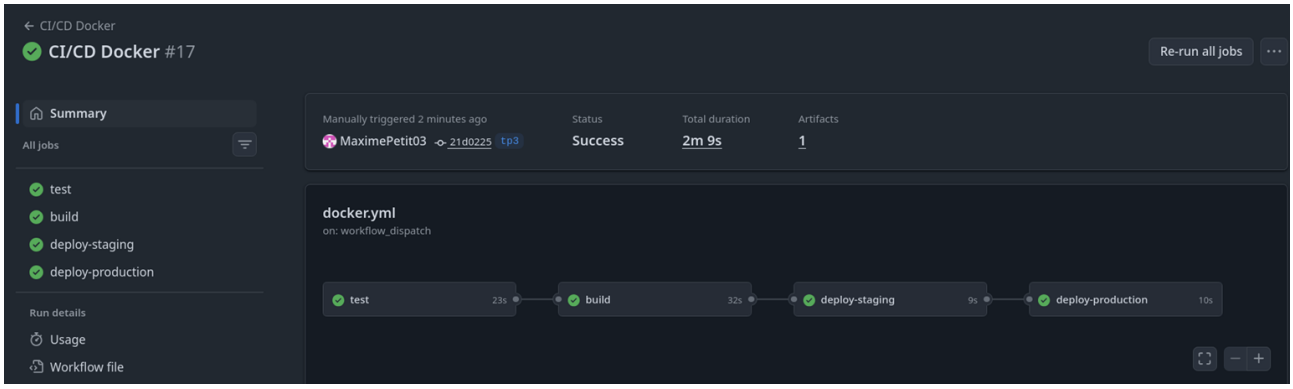

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ docker run --rm -p 8000:8000 ghcr.io/maximepetit03/tasks-api:0.1.0
Unable to find image 'ghcr.io/maximepetit03/tasks-api:0.1.0' locally
0.1.0: Pulling from maximepetit03/tasks-api
Digest: sha256:9f392e8508cffd458a9d6708f7d29d86bfe3f6c73298714c1497a574b200ba70
Status: Downloaded newer image for ghcr.io/maximepetit03/tasks-api:0.1.0
[2026-11-04 10:52:16 +0000] [1] [INFO] Starting gunicorn 26.2.0
[2026-11-04 10:52:16 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-11-04 10:52:16 +0000] [1] [INFO] Using worker: sync
[2026-11-04 10:52:16 +0000] [7] [INFO] Booting worker with pid: 7
[2026-11-04 10:52:16 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
```

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8000/version
{"version":"0.1.0"}
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Question :
Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?
Pour que ça marche sur plusieurs machines et pas seulement sur la mienne. Pour avoir un résultat commun

Partie 5 :





Questions : À quoi sert un test de fumée ? Un test de fumée vérifie rapidement que l'application démarre et répond sur ses routes essentielles avant d'aller plus loin
Pourquoi demander une approbation humaine avant la production ? L'approbation humaine apporte une validation finale et sécurisée pour maîtriser le moment et l'impact de la mise en production

Partie 6 :

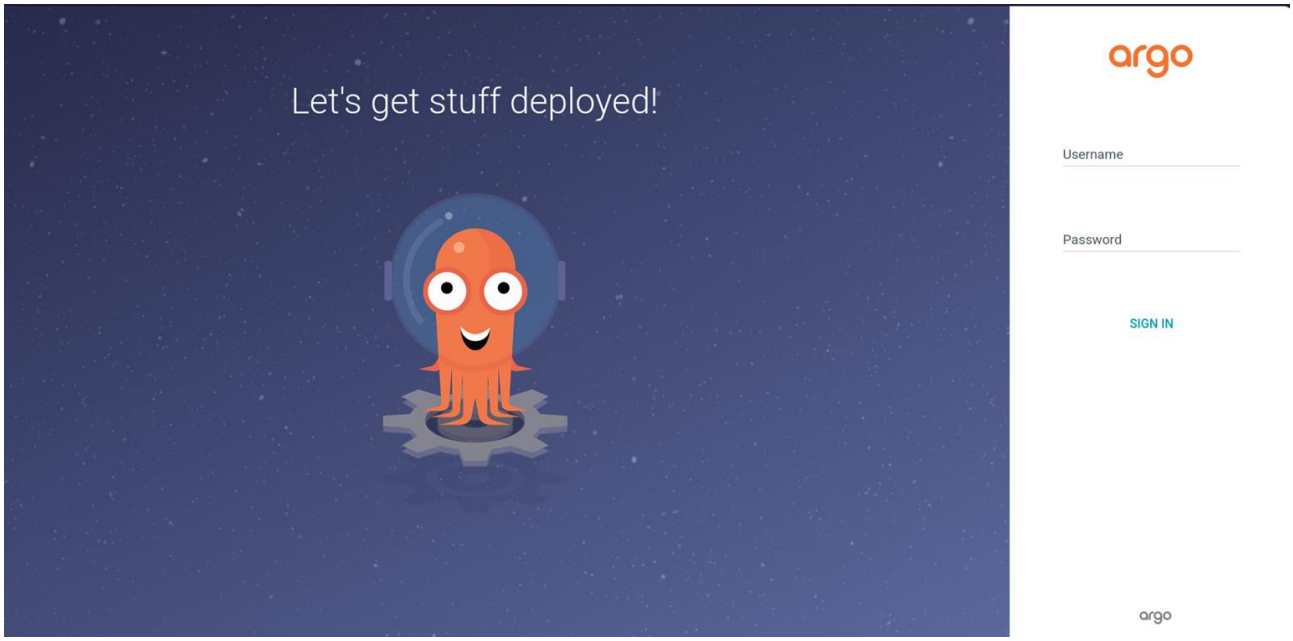
```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ docker run --rm tasks-api:local id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Question : Pourquoi ne pas exécuter une application en root dans un conteneur ? Exécuter une application en root dans un conteneur est dangereux parce qu'en cas de faille un attaquant qui s'échappe du conteneur obtient immédiatement les privilèges d'administration sur le serveur

TP4 :

Partie 1 :

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   35m   v1.37.0
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

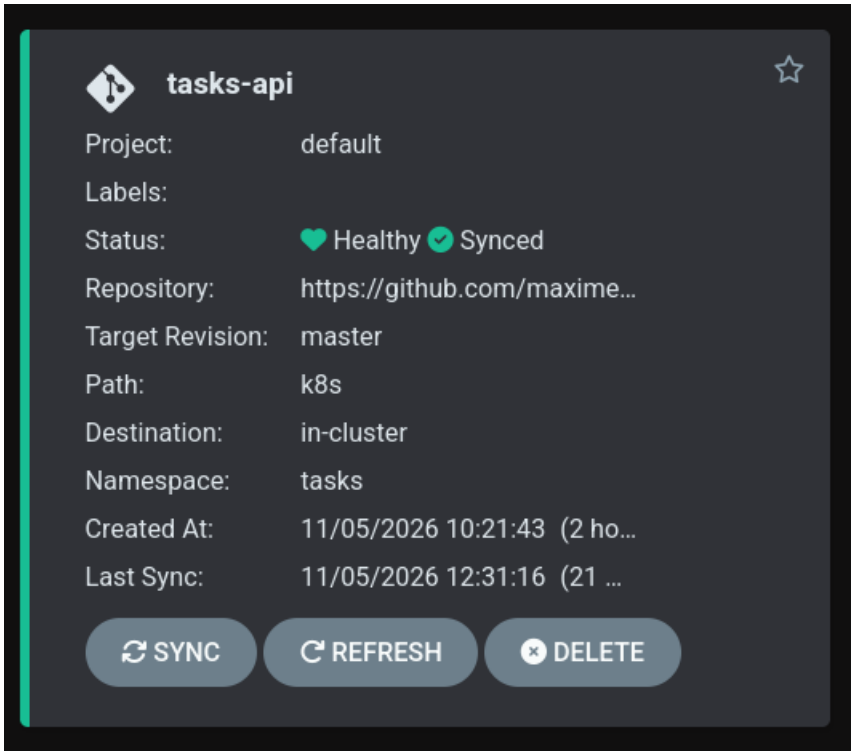



```
^Cmaxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          6m13s
argocd-applicationset-controller-56b6bc6dfc-khz87   1/1     Running   0          6m13s
argocd-dex-server-58f7b97876-pwf2t                  1/1     Running   0          6m13s
argocd-notifications-controller-7554d444c5-4jpg9    1/1     Running   0          6m13s
argocd-redis-b459689dd-dtfp6                        1/1     Running   0          6m13s
argocd-repo-server-5bbf479748-2t4v2                 1/1     Running   0          6m13s
argocd-server-f8cbdcd47-b5rnc                       1/1     Running   0          6m13s
```

Questions : C'est quoi un pod ? C'est le plus petit composant exécutable dans Kubernetes regroupant un ou plusieurs conteneurs qui partagent le même réseau et le même stockage
Un deployment ? C'est un contrôleur qui gère le cycle de vie de vos Pods permettant de définir le nombre de répliques souhaité de faire des mises à jour sans interruption

Pourquoi un namespace ? Il sert à diviser virtuellement un cluster Kubernetes en plusieurs espaces isolés afin d'organiser les ressources de séparer les environnements et d'appliquer des règles de sécurité distinctes

Partie 2 :



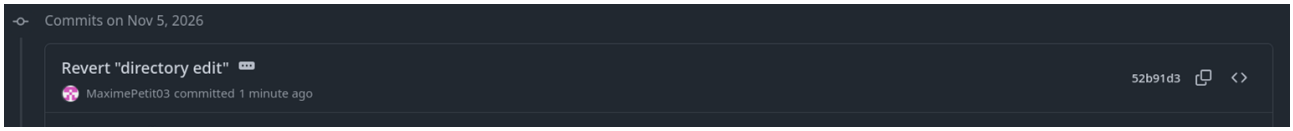

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8081/version
{"version":"0.1.0"}
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Question : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?
Séparer le code source et la configuration permet de sécuriser les accès d'appliquer des changements d'infrastructure instantanément sans recompiler l'application et de garantir une source unique pour le déploiement GitOps

Partie 3 :

```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-config$ kubectl get pods -n tasks
NAME                         READY   STATUS    RESTARTS   AGE
tasks-api-7966fd658c-gbgkv   1/1     Running   0          74m
tasks-api-7966fd658c-ktfvb   0/1     Running   0          1s
tasks-api-7966fd658c-x8hm7   1/1     Running   0          74m
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-config$
```

TP4-6 : J'ai vu les 5 pods pendant environ 5 à 10 secondes. Après ArgoCD est revenu à 3 pods



Questions : Que signifie selfHeal ? **selfHeal** est l'option d'ArgoCD qui annule automatiquement toute modification manuelle faite sur le cluster pour restaurer l'état exact défini dans Git.
Pourquoi le retour arrière est-il simple en GitOps ? Le retour arrière en GitOps est extrêmement simple car il suffit d'effectuer un git revert sur votre dépôt de configuration pour qu'ArgoCD redéploie instantanément la version précédente

Partie 4 :




```
maxime@Pc-portable-Maxime:~/Documents/GitOps/tasks-api$ curl http://localhost:8081/version
{"version":"0.2.0"}
maxime@Pc-pc-portable-Maxime:~/Documents/GitOps/tasks-api$
```

Questions :

Résumez en 5 lignes le chemin d'un git tag jusqu'au pod.

1. Le développeur crée et pousse un tag v0.2.0 sur le dépôt applicatif tasks-api
2. GitHub Actions se déclenche, teste le code, construit l'image Docker avec ce tag et la publie sur le registre GHCR
3. À la fin du pipeline, GitHub Actions met à jour le manifeste k8s/deployment.yaml dans le dépôt tasks-config avec le nouveau tag d'image
4. ArgoCD, qui surveille en permanence le dépôt tasks-config et détecte le changement de version du manifeste
5. ArgoCD applique la modification sur le cluster Kubernetes qui télécharge la nouvelle image depuis GHCR et relance le Pod avec la version à jour

Qui pousse (push) et qui tire (pull) ? GitHub Actions pousse la nouvelle image sur GHCR et le commit dans tasks-config alors qu'ArgoCD tire la configuration depuis Git et le cluster Kubernetes tire l'image depuis GHCR

Quelle différence avec un déploiement CI/CD « classique » ? Dans un CI/CD classique c'est le pipeline CI qui se connecte directement au cluster via kubectl pour pousser le déploiement alors qu'en GitOps c'est un agent interne au cluster qui surveille Git et tire les changements de configuration garantissant l'auto-guérison et la réconciliation continue